Hierarchical Latent Prediction for Language Models

Chang Shi[1]*, **Tim Pearce**[2], **Manan Tomar**[2], **Siddhartha Sen**[2], **John Langford**[2]
[1]University of Texas at Austin, [2]Microsoft Research

arXiv:2608.05806v1 [cs.CL] 6 Aug 2026

Abstract

While standard Next-Token Prediction (NTP) lays the foundation of language model pretraining, its teacher-forced training paradigm may not be optimal for long-horizon reasoning and planning. Recent works such as Multi-Token Prediction (MTP) and Next-Latent prediction (NextLat) try to mitigate the problem through predicting multiple future tokens and self-supervised prediction in the latent space. However, those auxiliary objectives either have a limited horizon or suffer from compounding error from multi-step rollout. We introduce **Hierarchical Latent Prediction** (HiLP), which introduces an auxiliary higher-level abstract latent to help reduce the error accumulation effect in latent-space rollouts. Experiments show that HiLP can lead to longer-horizon coherent belief state representation and demonstrate the effectiveness of our method across coding and multi-step reasoning benchmarks, and offers more speculative decoding efficiency.

1 Introduction

Next-token prediction (NTP) with teacher forcing has yielded transformers of remarkable capability, yet it also introduces a training-inference mismatch known as exposure bias: during inference time, the auto-regressive token generation relies on its own previous outputs, leading to compounding errors which could degrade long-range generation quality.

Recent work NextLat (Teoh et al., 2025) partially mitigates the issue through enforcing a latent space self-prediction loss. However, its latent dynamics model predicts one step ahead. This means that the self-predictive learning signal, while effective in inducing local transition consistency, provides only indirect gradient pressure to form representations that capture structure at longer temporal scales. In practice, long-horizon dependencies still remain attenuated by sequential dynamics unrolling.

Multi-scale temporal abstraction methods have a long history in sequence modeling and reinforcement learning (Sutton et al., 1999; Bacon et al., 2017). Inherently, language incorporates hierarchical structures at multiple scales, from characters, phrases to sentences, paragraphs and discourses. Therefore, it is reasonable to extend NextLat with hierarchical learning signals at different granularities, for the sake of richer latent transition dynamics. This observation motivates a natural question: *can we introduce temporal hierarchy directly into the latent-space self-prediction objective, shaping the representation of a language model transformer to encode multi-scale predictive structure during pretraining?*

In this paper, we answer affirmatively by introducing **Hierarchical Latent Prediction (HiLP)**. Our contributions are as follows:

1. We propose HiLP, a hierarchical representation training method that introduces multi-scale self-predictive learning into transformer pretraining via sliding-window attention over latents, a higher-level dynamics model, and a combined NTP head.

2. We evaluate the resulting models on downstream benchmark accuracy and speculative decoding efficiency, comparing HiLP with MTP and NextLat.

3. We demonstrate that the entire hierarchical apparatus can be removed at inference time with no architectural overhead, preserving NextLat's option of being a pure training-time intervention.

2 Methodology

We use $X_{1:T}$ to denote the token sequence prefix. A transformer G_θ produces *lower-level* la-

* Work done during internship at Microsoft Research. Correspondence to: Chang Shi <changshi@utexas.edu>

tents $\mathbf{h}_t = G_\theta(X_{1:t})$. A sliding-window attention (SWA) module with window k produces *higher-level* abstract latents $\mathbf{u}_t = \mathrm{SWA}(\mathbf{h}_{t-k+1:t})$, a deterministic function of the last k lower states.

A higher-level dynamics model is trained to predict k steps ahead in this abstract space. This explicit coarser-scale prediction objective encourages lookahead planning beyond single-step latent transitions. A combined NTP head then conditions the next-token prediction on both level latents, taking advantage of abstract lookahead information to improve subsequent predictions. Four parameterised maps are jointly learned: output head p_θ, lower dynamics p_ψ, upper dynamics p_ϕ, and combined head p_ρ. An illustration of the architecture is shown in Fig. 1.

Training objectives. The total training objective is then a weighted sum of five terms: the standard NTP loss, the NextLat (lower-level) transition consistency loss, the KL term, the higher-level transition consistency loss, and the combined NTP loss. At inference time, only the standard NTP head is used, the entire hierarchical apparatus serves purely as an auxiliary training signal.

$$\mathcal{L}_{\text{ntp}} = \mathbb{E}_{t<T}[-\log p_\theta(X_{t+1} \mid \mathbf{h}_t)] \quad \text{(A)}$$

$$\mathcal{L}_{\text{h}} = \mathbb{E}_t\left[\tfrac{1}{d}\textstyle\sum_{i=1}^{d} \mathrm{SL1}(\mathrm{sg}[\mathbf{h}_{t+i}], \hat{\mathbf{h}}_{t+i})\right] \quad \text{(B)}$$

$$\mathcal{L}_{\text{KL}} = \mathbb{E}_t\Big[\tfrac{1}{d}\textstyle\sum_{i=1}^{d} D_{\text{KL}}\big(p_\theta^{\text{sg}}(\cdot \mid \mathrm{sg}[\mathbf{h}_{t+i}]) \,\big\|\, p_\theta^{\text{sg}}(\cdot \mid \hat{\mathbf{h}}_{t+i})\big)\Big] \quad \text{(C)}$$

$$\mathcal{L}_{\text{u}} = \mathbb{E}_t[\mathrm{SmoothL1}(\mathrm{sg}[\mathbf{u}_{t+k}],\ \hat{\mathbf{u}}_{t+k})] \quad \text{(D)}$$

$$\mathcal{L}_{\text{cntp}} = \mathbb{E}_{t<T}\big[- \log p_\rho(X_{t+1} \mid \mathbf{h}_t, \tilde{\mathbf{u}}_t)\big] \quad \text{(E)}$$

Where $\mathrm{SL1}(\cdot,\cdot)$ denotes the SmoothL1 loss, $\mathrm{sg}[\cdot]$ the stop-gradient operator, and $\tilde{\mathbf{u}}_t := \mathrm{SWA}(\mathrm{sg}[\mathbf{h}_{t-k+1:t}])$ the higher-level latent with sg applied inside SWA so gradients do not flow back into $\mathbf{h}$. We use this stop-gradient in the combined NTP path to keep the token-level representation $\mathbf{h}_t$ governed by the standard NTP and lower-level transition consistency losses, while allowing the combined head to train the SWA module and its higher-level representation. Without this separation, the combined-head cross-entropy would also update the lower latents through the SWA window, double-counting token-level supervision and creating a competing optimization signal for $\mathbf{h}_t$.

Consistency conditions. At optimality, the losses enforce:

$$p_\theta(X_{t+1} \mid \mathbf{h}_t) = \mathbb{P}(X_{t+1} \mid X_{1:t}) \quad \text{(I)}$$

$$p_\psi(\mathbf{h}_{t+1} \mid \mathbf{h}_t, X_{t+1}) = \mathbb{P}(\mathbf{h}_{t+1} \mid X_{1:t+1}) \quad \text{(II)}$$

$$p_\phi(\mathbf{u}_{t+k} \mid \mathbf{u}_t) = \mathbb{P}(\mathbf{u}_{t+k} \mid X_{1:t}) \quad \text{(III)}$$

$$p_\rho(X_{t+1} \mid \mathbf{h}_t, \mathbf{u}_t) = \mathbb{P}(X_{t+1} \mid X_{1:t}) \quad \text{(IV)}$$

where (I) and (IV) enforce next-token consistency, (II) and (III) enforce transition consistency.

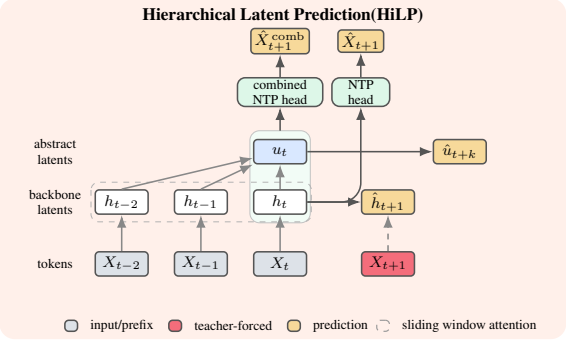


Figure 1: Overview of HiLP with sliding-window size $W = 3$. The higher-level latent is computed by sliding-window attention over backbone latents, the higher next-latent predictor maps u_t to $\hat{u}_{t+W}$ without token input, and both the standard NTP head and combined NTP head are trained, but the combined NTP head is dropped during inference to ensure no slow down in inference speed.

3 Experiments

1B-parameter models are trained on 100B tokens using 8 × NVIDIA B200 GPUs.

3.1 Coding and Multi-step Reasoning Benchmarks

After pretraining, we use LM Evaluation Harness (Gao et al., 2024) to evaluate the zero-shot performance of the models on HumanEval coding benchmark (Chen et al., 2021), and DataComp for LLMs (Li et al., 2025) to evaluate the models on a set of symbolic and multi-step benchmarks. Results in Tab. 1 and Fig. 2 show that HiLP is improve performance from the multi-scale latent prediction.

3.2 Speculative Decoding

We evaluate speculative decoding on held-out validation splits of code and Nemotron-Climbmix data(Diao et al., 2026). For each model, we report the **average number of accepted tokens** per drafting step, and **Avg match**, the per-position draft-verifier argmax agreement averaged over K=1. . .4.

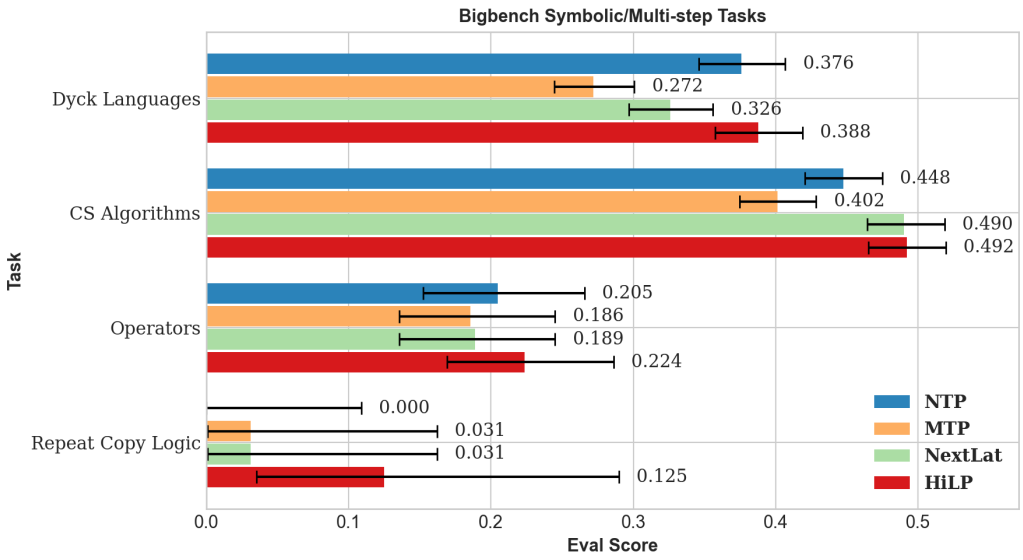


Figure 2: DCLM symbolic/multi-step eval results

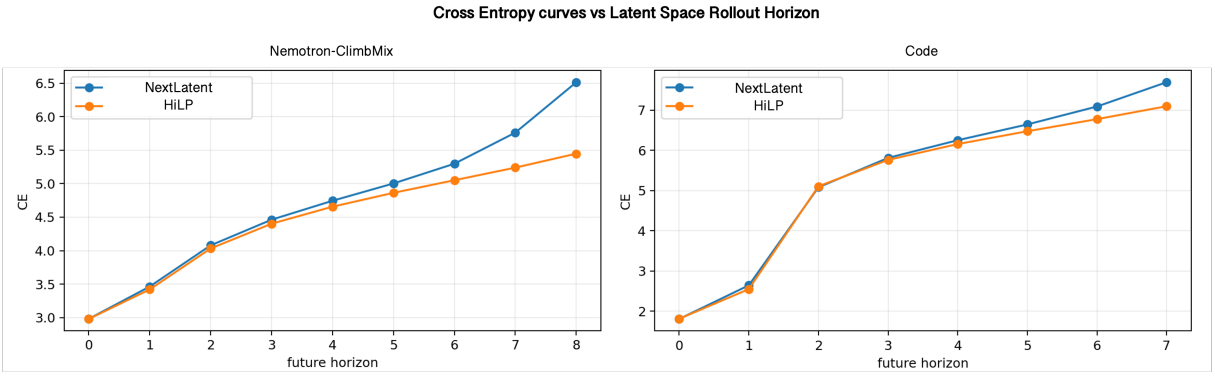


Figure 3: Latent cross-entropy curves for NextLat and HiLP on web and code splits. HiLP has comparable near-term latent prediction loss and lower long-horizon future and excess cross-entropy, indicating more stable multi-step latent rollout.

Model	HumanEval pass@1
NTP	8.77
MTP	9.21
NextLat	10.58
HiLP	11.33

Table 1: Code-generation benchmark accuracy on HumanEval. HumanEval results are computed from 1000 samples per task.

Note that $K{=}0$ NTP position is always accepted and is omitted from the per-K columns;

4 Discussion

Longer horizon prediction. The latent prediction cross-entropy losses across rollout horizons up to 8 steps ahead are shown in 3. As the curves show, HiLP preserves the near-term cross-entropy of NextLat while producing lower future prediction error at longer horizons, suggesting that the hierarchical latent mitigates multi-step error accumulation.

5 Related Work

Our work sits at the intersection of several active research threads: pretraining objectives beyond next-token prediction, latent-space language modeling, and hierarchical models.

5.1 Pretraining Objectives Beyond Next-Token Prediction

The next-token prediction objective has largely limited the capacity of language models in downstream tasks that require longer-horizon reasoning and planning(Bachmann and Nagarajan, 2024; Nagarajan et al., 2025). Recent works have started to introduce auxiliary learning signals to mitigate

Model	Avg Acc. Tok.↑	Avg match↑	K=1	K=2	K=3	K=4	K=5	K=6	K=7	K=8
MTP	2.57	0.545	0.707	0.633	0.531	0.310	–	–	–	–
NextLat	3.26	0.580	0.918	0.652	0.422	0.328	0.309	0.285	0.234	0.234
HiLP (NTP head)	3.41	0.592	0.938	0.645	0.414	0.367	0.324	0.324	0.293	0.340

Table 2: Speculative decoding results evaluated on a held-out validation split of code data.

Model	Avg Acc. Tok.↑	Avg match↑	K=1	K=2	K=3	K=4	K=5	K=6	K=7	K=8
MTP	2.11	0.162	0.301	0.141	0.109	0.098	–	–	–	–
NextLat	2.57	0.483	0.641	0.504	0.410	0.375	0.324	0.312	0.246	0.254
HiLP (NTP head)	2.59	0.491	0.699	0.492	0.379	0.395	0.328	0.406	0.289	0.289

Table 3: Speculative decoding results evaluated on a held-out validation split of Nemotron-ClimbMix data.

this myopic gap through further future prediction (Gloeckle et al.; Ahn et al., 2025; Teoh et al., 2025; Mahajan et al., 2025). However, these approaches usually operate on a single granularity scale and have limited horizon capability due to compounding error. HiLP differs in two ways: it introduces two latent prediction pathways at different temporal scales, so the coarser pathway supervises long-horizon structure directly rather than through repeated single-step unrolling; and all auxiliary heads are dropped at inference, so the longer-horizon signal costs nothing at deployment time.

5.2 Latent-Space Language Modeling

Another line of work moves language generation itself into a continuous latent space. Large Concept Models (Barrault et al., 2024) perform autoregressive prediction over sentence-level embeddings, treating each sentence as a "concept" in a shared representation space. CALM (Shao et al., 2025) replaces next-token prediction with next-vector prediction, compressing a chunk of K tokens into a single continuous vector to raise the semantic bandwidth of each generative step. Coconut (Shao et al., 2025) lets the model reason in latent space by feeding its own hidden state back as the next input embedding instead of decoding to tokens. All of these approaches change the inference-time generation process to operate on latents, requiring bespoke decoding procedures or latent-to-text decoders. HiLP is the opposite design point: latent prediction is used purely as an auxiliary training signal to shape representations, while inference remains standard token-level autoregressive decoding with zero added latency.

5.3 Hierarchical Models

Hierarchical model design has been researched in both language modeling area and other domains.

MegaByte (Yu et al., 2023) stacks a global patch-level transformer over a local byte-level one, the Byte Latent Transformer (Pagnoni et al., 2025) dynamically segments bytes into entropy-based patches that serve as the units of computation, and H-Net (Hwang et al., 2025) learns content-dependent chunking end-to-end within a hierarchical U-Net-like network.

In model-based reinforcement learning, Hierarchical Planning with Latent World Models (Zhang et al., 2026) learns world models at multiple temporal scales within a shared latent space, using long-horizon latent predictions as subgoals for short-horizon control. In representation learning theory, Learning Discrete Concepts in Latent Hierarchical Models (Kong et al., 2024) formalizes concepts as discrete latent variables organized in a hierarchical causal model and derives identifiability conditions for recovering such hierarchical concept structure from high-dimensional unsupervised data. In time series analysis. HiTime (Tao et al., 2024) employs a hierarchical feature encoder together with a hybrid prompting strategy to align time series and text modalities, improving multivariate time series classification with large language models. These works demonstrate the benefit of hierarchical latent structure across planning, representation identifiability, and time series domains. HiLP differs in a way that the hierarchy complexity is not consumed at inference time: to our knowledge it is the first to introduce hierarchical latent prediction as a pretraining objective for autoregressive language models, where the hierarchy shapes the representation during training and is then removed entirely.

6 Conclusion

HiLP introduces temporal hierarchy into the latent-space self-prediction objective, shaping the representation of language model transformers to encode multi-scale predictive structure during pretraining. Experiments show that it improves both speculative